

DEPLOYMENT AND OPERATIONS GUIDE

ESS Portal

Production Setup, CI/CD and Automatic Cloud Deployment

UPDATED PURPOSE

This revision explains how a developer can update the local codebase, push an approved Git revision, and have the cloud server update automatically through GitHub Actions. It also retains the server setup, backup, rollback, monitoring and security controls required for production.

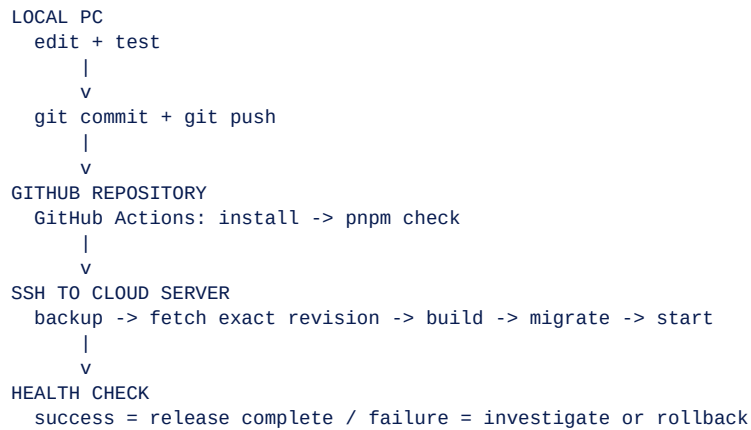
Document item	Value
Repository	businesssta/Company-Portal
Updated date	20 August 2026
Primary automation	GitHub Actions + SSH + Docker Compose
Staging trigger	Push to develop
Production trigger	Push/merge to main after enabling the documented workflow change
Document status	Updated automatic deployment handover

IMPORTANT

Saving a file on localhost alone cannot update the cloud server. The automatic trigger occurs only after the change is committed and pushed to the configured Git branch. Git remains the release record and rollback reference.

1. Desired Update Flow

The intended operational flow is shown below. The cloud server never reads files directly from the developer's computer.



1.1 Branch behavior

Branch	Purpose	Recommended behavior
feature/*	Individual development	Validation through pull request; no cloud deployment
develop	Shared testing	Automatic staging deployment after validation
main	Live release	Automatic production deployment after validation; protect main with pull-request review

1.2 What 'automatic' means

- The developer tests locally and pushes a commit.
- GitHub Actions validates the exact commit before any server change.
- The workflow connects to the server using a dedicated SSH key.
- The server creates backups, rebuilds containers, runs pending migrations and checks health.
- The workflow result becomes the deployment audit trail.

Do not configure a cloud server to continuously copy an uncommitted localhost folder. That removes review, audit and reliable rollback.

2. Current Repository Readiness

The repository already includes the deployment building blocks below. The principal change needed for automatic production deployment is the production job trigger in the GitHub Actions workflow.

Repository file	Purpose
docker-compose.yml	Runs PostgreSQL, API, administration web and employee web with isolated project names and persistent volumes.
deploy/staging.env.example	Safe staging defaults and isolated ports.
deploy/production.env.example	Safe production defaults and isolated ports.
deploy/deploy.sh	Backs up data, fetches an exact revision, rebuilds, starts services and performs health checks.
.github/workflows/deploy.yml	Validates pushes and deploys over SSH. Staging is currently automatic; production is currently manual until Section 6 is applied.
DEPLOYMENT.md	Repository-level operational runbook.

2.1 Controls already provided

- `pnpm install --frozen-lockfile` and `pnpm check` run before deployment.
- The remote deployment script creates database and upload backups when services are running.
- The selected Git revision is deployed rather than an untracked server copy.
- Pending database migrations run during API startup.
- The deployment fails when the web health endpoint does not become healthy.

3. Production Topology and Isolation

ESS Portal can share an Ubuntu server with the Meeting Room application only when domains, ports, Docker project names, databases, networks and volumes remain separate.

Environment	Example URL	Host port	Git branch
ESS staging admin	dev-ess.example.com	8180	develop
ESS staging employee	dev-employee.example.com	8181	develop
ESS production admin	ess.example.com	8280	main
ESS production employee	employee.example.com	8281	main

3.1 Server prerequisites

- Current Ubuntu LTS, Docker Engine 26 or newer, Docker Compose v2, Git and curl.
- At least 2 CPU cores and 4 GB RAM, adjusted for other applications on the host.
- DNS control, trusted HTTPS certificates and outbound access to GitHub/container registries.
- Dedicated non-root deployment account with permission to run Docker.

3.2 Conflict checks

```
sudo ss -lntp
docker ps --format 'table {{.Names}}\t{{.Ports}}\t{{.Status}}'
docker network ls
docker volume ls
df -h
free -h
```

STOP CONDITION: do not continue when an ESS port is already in use. Select a different port and update the matching environment file and reverse proxy target.

4. One-Time Ubuntu Setup

4.1 Create isolated working copies

```
sudo mkdir -p /opt/ess-portal-staging /opt/ess-portal-production
sudo chown -R DEPLOY_USER:DEPLOY_USER /opt/ess-portal-staging /opt/ess-portal-production

git clone --branch develop https://github.com/businesssta/Company-Portal.git /opt/ess-portal-staging
git clone --branch main https://github.com/businesssta/Company-Portal.git /opt/ess-portal-production
```

4.2 Create protected environment files

```
cd /opt/ess-portal-staging
cp deploy/staging.env.example .env
chmod 600 .env

cd /opt/ess-portal-production
cp deploy/production.env.example .env
chmod 600 .env
```

4.3 Required separation

- Use different COMPOSE_PROJECT_NAME values.
- Use different databases, database users, passwords and JWT secrets.
- Use different loopback ports and public domains.
- Never commit .env, private keys, database dumps or upload archives.

4.4 Generate secrets

```
openssl rand -hex 64
openssl rand -base64 36
```

5. GitHub CI/CD Secrets

5.1 Create environments

Open GitHub Repository Settings -> Environments. Create **staging** and **production**.

Secret	Required value
DEPLOY_HOST	Ubuntu hostname or public/private IP reachable by GitHub Actions
DEPLOY_USER	Dedicated Ubuntu deployment account
DEPLOY_SSH_KEY	Private key for the deploy account
DEPLOY_KNOWN_HOSTS	Independently verified SSH host-key entry

Variable	Staging	Production
DEPLOY_PORT	22 or approved SSH port	22 or approved SSH port
DEPLOY_PATH	/opt/ess-portal-staging	/opt/ess-portal-production

5.2 Create a deploy key

```
ssh-keygen -t ed25519 -f ess_portal_deploy -C ess-portal-deploy
ssh-keyscan -H YOUR_SERVER_HOST
```

- Install only the .pub key in DEPLOY_USER's ~/.ssh/authorized_keys.
- Store the private key only in GitHub Environment secrets.
- Verify the host key through a trusted channel before saving DEPLOY_KNOWN_HOSTS.

6. Enable Automatic Production Deployment

The current workflow deploys production only through workflow_dispatch. To make an approved main push update the cloud server automatically, change the deploy-production job condition in `.github/workflows/deploy.yml`.

6.1 Replace the current condition

```
# CURRENT - manual production only
if: github.event_name == 'workflow_dispatch' && inputs.target == 'production'
```

6.2 Use this condition

```
# UPDATED - every push/merge to main deploys production
if: >-
  (github.event_name == 'push' && github.ref_name == 'main') ||
  (github.event_name == 'workflow_dispatch' && inputs.target == 'production')
```

6.3 Deploy the exact pushed revision

The production DEPLOY_REVISION expression should keep the pushed commit SHA for push events and origin/main for a manual run:

```
DEPLOY_REVISION: ${ { github.event_name == 'workflow_dispatch' && 'origin/main' || github.sha } }
```

6.4 Protect main before enabling auto production

- Require a pull request before merging to main.
- Require the validation status check to pass.
- Block force pushes and direct deletion of main.
- Require at least one reviewer for production-impacting changes.

If the production GitHub Environment has a required reviewer, the workflow starts automatically but waits for approval. Remove that reviewer rule only when fully unattended production deployment is explicitly accepted.

7. Daily Local-to-Cloud Procedure

7.1 Test locally

```
pnpm install --frozen-lockfile
pnpm check
pnpm dev
```

Confirm the changed screens and API behavior at localhost before creating the release commit.

7.2 Update staging automatically

```
git switch develop
git pull --ff-only
git add <reviewed-files>
git commit -m "Describe the ESS Portal change"
git push origin develop
```

GitHub Actions validates the push and deploys the exact commit to staging. Test the staging URLs; localhost is no longer involved after the push.

7.3 Update production automatically

- Create a pull request from develop to main.
- Wait for validation and staging acceptance.
- Review and merge the pull request.
- The resulting push to main starts the production deployment automatically.
- If an Environment approval gate is configured, approve it in GitHub Actions.
- Verify the production URL, login and the changed business workflow.

7.4 Direct main workflow - not recommended

```
git switch main
git pull --ff-only
# merge only reviewed work
git push origin main
```

A direct main push is safe only when branch protection, review and organizational policy explicitly allow it.

8. What Happens on the Cloud Server

GitHub Actions uses SSH to run `deploy/deploy.sh` inside the correct deployment directory. The script performs these operations in order:

- Loads the protected server `.env` file.
- Creates timestamped PostgreSQL and upload backups when containers are running.
- Fetches the named branch and resets the working copy to the requested exact revision.
- Validates Docker Compose configuration.
- Rebuilds images and starts/recreates services without deleting persistent volumes.
- Allows the API startup process to apply pending migrations.
- Waits for the web health endpoint and prints diagnostics on failure.

8.1 First manual verification

```
cd /opt/ess-portal-production
docker compose --project-name ess-portal-production --env-file .env config --quiet
bash deploy/deploy.sh main origin/main
curl --fail http://127.0.0.1:8280/health
curl --fail http://127.0.0.1:8280/healthz
```

Complete one successful manual production deployment before switching on automatic main deployment. This validates credentials, paths, ports, Compose configuration, backups and health checks.

9. Reverse Proxy, DNS and HTTPS

Nginx or Caddy should accept public traffic on ports 80 and 443. ESS containers should remain bound to 127.0.0.1.

9.1 Nginx pattern

```
server {
    listen 80;
    server_name ess.example.com;
    location / {
        proxy_pass http://127.0.0.1:8280;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

9.2 Validate

```
sudo nginx -t
sudo systemctl reload nginx
```

- Create DNS records before requesting certificates.
- Issue trusted TLS certificates for every hostname and redirect HTTP to HTTPS.
- Set WEB_ORIGIN to the exact HTTPS browser origins without trailing slashes.
- Do not expose PostgreSQL or the API container port publicly.

10. Monitoring and Transaction Logs

10.1 Server status

```
cd /opt/ess-portal-production
docker compose --project-name ess-portal-production --env-file .env ps
docker compose --project-name ess-portal-production --env-file .env logs --tail=200 api db web mobile
curl --fail http://127.0.0.1:8280/health
curl --fail http://127.0.0.1:8280/healthz
```

10.2 Live API transaction monitoring

```
docker compose --project-name ess-portal-production --env-file .env logs -f --tail=100 api
```

Press Ctrl+C to stop following logs; the containers continue running. Avoid exposing passwords, tokens, personal data or full request bodies in production logs.

10.3 Minimum alerts

- Admin and employee HTTP health checks.
- Container restart/unhealthy-state alerts.
- Disk usage alerts for Docker, PostgreSQL, uploads and backups.
- Backup success and periodic restore-test records.
- TLS certificate expiry monitoring and retained API/reverse-proxy logs.

11. Backup, Rollback and Failure Handling

11.1 Backup policy

- Copy deployment backups to separate storage; local server copies are not sufficient.
- Encrypt backups at rest and in transit and define daily/weekly/monthly retention.
- Back up database and uploads as a matching recovery set.
- Test restoration regularly.

11.2 Application rollback

```
cd /opt/ess-portal-production
git log --oneline -10
bash deploy/deploy.sh main PREVIOUS_COMPATIBLE_COMMIT_SHA
```

11.3 Failed automatic deployment

- Open the failed GitHub Actions run and identify whether validation, SSH, build or health check failed.
- Check API, database and web logs on the server.
- Do not repeatedly rerun a deployment without understanding a migration or data failure.
- Fix forward when safe; otherwise deploy the previous compatible revision.

Never run docker compose down -v in staging or production. The -v option deletes persistent database and upload volumes. Do not roll code back across an incompatible migration without restoring the matching pre-deployment backup.

12. Security and Acceptance Checklist

12.1 Security baseline

- Use a dedicated non-root deployment account and a dedicated SSH key.
- Restrict inbound SSH to approved sources where possible; publish only HTTPS.
- Keep databases and internal API networking private.
- Use unique staging and production credentials and rotate them under change control.
- Protect main with reviews and required CI checks before unattended production deployment.
- Patch Ubuntu, Docker and base images on a schedule.

12.2 Go-live checklist

- ☐ Meeting Room remains healthy and unchanged.
- ☐ All ESS ports are confirmed unused.
- ☐ Staging and production use separate Compose projects, databases and volumes.
- ☐ Protected .env files exist with chmod 600 and are not tracked by Git.
- ☐ DNS and trusted HTTPS certificates are active.
- ☐ GitHub secrets, variables and host keys are verified.
- ☐ A manual first deployment and health check have passed.
- ☐ develop automatically updates staging.
- ☐ main branch protection is active before automatic production deployment.
- ☐ Database/upload backups are verified and copied off-server.
- ☐ Production login, core workflows, uploads, imports/exports and printing pass.
- ☐ Monitoring, retention and incident ownership are assigned.

Appendix A. Server Values Worksheet

Configuration item	Approved value / owner / date
Ubuntu host / IP	
SSH port	
Deploy username	
Meeting Room occupied ports	
Staging domains / ports	
Production domains / ports	
Backup destination and retention	
Main branch reviewer	
Production approval policy	
Monitoring owner	
Planned go-live window	

Appendix B. Release Record

Field	Value
Release commit SHA	
Pull request	
Staging validation date	
Production trigger time	
Database backup file	
Upload backup file	
GitHub Actions run	
Post-deployment verifier	

Appendix C. Reference Locations

- Repository: <https://github.com/businesssta/Company-Portal>
- Deployment workflow: `.github/workflows/deploy.yml`
- Remote deployment script: `deploy/deploy.sh`
- Docker configuration: `docker-compose.yml`
- Repository runbook: `DEPLOYMENT.md`